

贡柑病虫害智能检测系统

项目计划书

队伍人数：2 人

开发周期：2 周

2026 年 7 月 3 日

目 录

1 项目概述	1
1.1 项目目标	1
1.2 最终交付物	1
1.3 核心功能优先级	1
2 技术方案总览	2
2.1 系统架构	2
2.2 技术选型	2
2.3 与已有经验的对比	2
3 识别目标: 5 类	3
3.1 分类列表	3
3.2 类别选择说明	3
4 数据集策略: 两次训练	4
4.1 第一轮训练 (Week 1-2 完成) ——基于公开数据	4
4.2 第二轮微调 (Week 3+) ——基于实地数据	4
4.3 备用方案 (无法获取本地数据时)	5
5 设备需求	5
5.1 关于无人机的补充说明	5
6 两人分工与两周时间线	6
6.1 角色划分	6
6.2 Week 1 (考试周, 每天 ≤ 2 小时)	6
6.3 Week 2 (冲刺周, 全力投入)	7
6.4 分工并行度分析	8
7 加分项路线图	8
7.1 未来展望 (写在报告/答辩中, 不实际开发)	9
8 风险分析与应对	9
9 待与老师 / 队友确认的问题	9

第1章 项目概述

1.1 项目目标

开发一款可在手机上使用的病虫害识别程序（Web APP），用户拍摄贡柑叶片或果实的照片后，系统自动识别病害类型，并给出对应的防治建议。

1.2 最终交付物

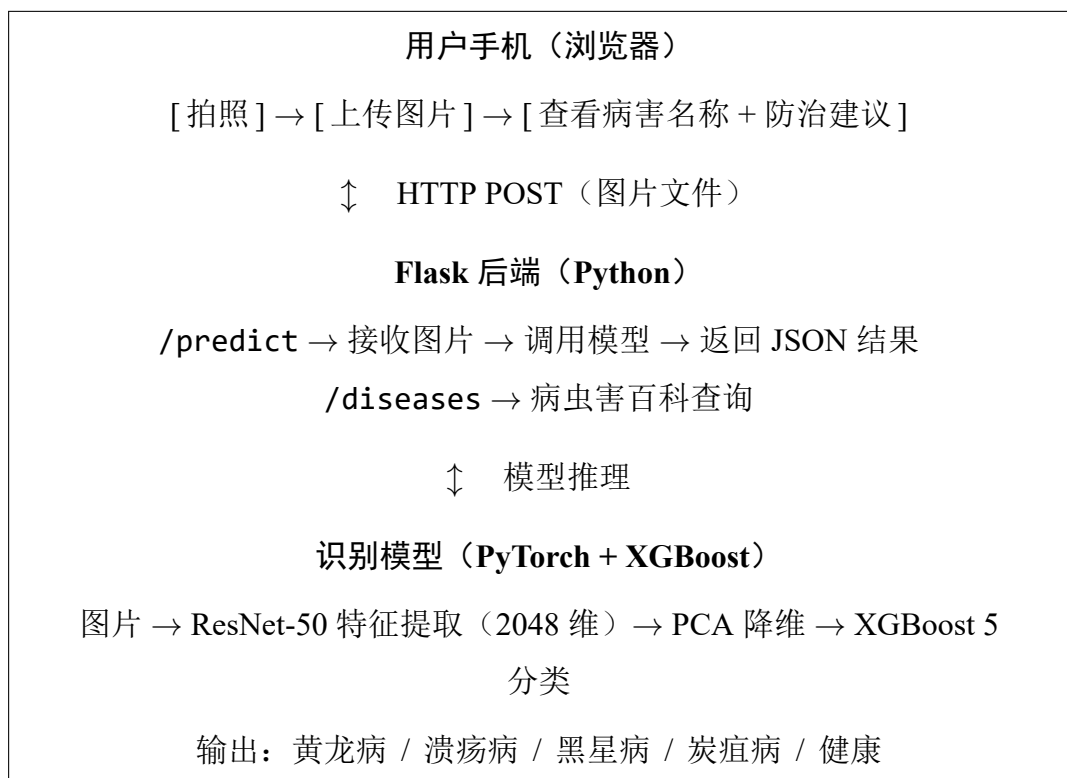
- 手机浏览器打开网页 → 拍照 → 出病害名称 + 防治方案
- 支持 PWA（Progressive Web App），可「添加到桌面」像原生 APP 一样使用
- 包含模型训练代码、评估报告，可后续迭代优化

1.3 核心功能优先级

优先级	功能	工作量	说明
P0（必须）	拍照识别病害	大	核心功能，模型训练 + 部署
P0（必须）	显示防治建议	小	JSON 数据文件，随结果返回
P0（必须）	移动端 Web APP	中	适配手机浏览器，调起相机
P1（加分）	防治建议含农药信息	小	扩充 JSON 数据
P1（加分）	无人机巡航模式演示	中	概念演示：航拍 → 颜色分析 → 标记异常区
P1（加分）	识别历史记录	小	浏览器 localStorage
P2（展望）	无人机实时视频流	大	下一阶段
P2（展望）	激光驱虫	大	写在「未来工作」，不实际开发

第2章 技术方案总览

2.1 系统架构



2.2 技术选型

层次	技术	选型理由
前端	HTML5 + CSS + JS	移动端适配，调起手机相机，PWA 桌面图标
后端	Python Flask	轻量级，快速开发，一行命令启动
模型	ResNet-50 + XGBoost	完全复用队伍已有的黄斑水肿项目经验
部署	本地电脑 / 后续上云	开发阶段本地运行，演示时手机同一 WiFi 访问

2.3 与已有经验的对比

队伍中的 A 同学此前完成过「视网膜病变识别——黄斑水肿危险程度分类」项目，使用 ResNet-50 特征提取 + XGBoost 分类器的技术路线。本项目在技术栈上高度复用，核心差异仅在于：

- 输入数据：从「眼底照片」换成「柑橘照片」

- 分类数：从「3 分类（风险 0/1/2）」换成「5 分类（4 种病害 + 健康）」
- 新增部署环节：模型训练完成后导出为 .pkl，由 Flask 后端加载

技术风险较低，主要工作量在数据准备和 APP 开发上。

第3章 识别目标：5 类

3.1 分类列表（最终分类根据数据集来）

编号	病害名称	严重程度	识别特征
0	黄龙病 (HLB)	毁灭性	叶片斑驳黄化、「红鼻果」、树势衰退
1	溃疡病 (Canker)	严重	叶片/果实隆起病斑、木栓化、黄色晕圈
2	黑星病 (Black Spot)	常见	叶片黑色小斑点、果实凹陷黑斑
3	炭疽病 (Anthracnose)	常见	叶片褐色枯斑、同心轮纹、潮湿时红色粘液
4	健康 (Healthy)	正常	正常绿色叶片/果实，无病斑虫痕

3.2 类别选择说明

- 5 类全部能与 Kaggle 公开柑橘数据集匹配，可零成本获取训练数据
- 虫害暂不纳入：虫害识别需要目标检测（定位虫子位置），比病害分类复杂一个量级，建议作为下一阶段功能
- 如果后续发现某类样本太少（<30 张），可临时合并或降为 4 分类

第4章 数据集策略：两次训练

4.1 第一轮训练 (Week 1–2 完成) ——基于公开数据

项目	说明
数据来源	Kaggle Citrus Leaves Dataset (Sourabh Sawant, 2024)
数据规模	~1,000 张, 含 Black Spot / Canker / Greening / Healthy / Melanose
类别匹配	Greening (黄龙病)、Canker (溃疡病)、Black Spot (黑星病)、Healthy (健康) Melanose 可归入黑星病类; 炭疽病需额外补充
训练方式	ResNet-50 提取 2048 维特征 → PCA 降维 → XG-Boost 5 分类
预期准确率	85%–95% (参考同类论文 EfficientNet-B5 达 99.22%, ResNet50 达 97%)
技术风险	低: 公开数据集成熟 + 队伍已有代码可直接改造

4.2 第二轮微调 (Week 3+) ——基于实地数据

项目	说明
数据来源	去当地贡柑果园拍摄, 每类病害 ≥50 张, 健康 ≥50 张
拍摄要求	不同光照 (早/中/阴)、不同角度、叶片正面 + 背面 + 果实
训练方式	在第一轮模型基础上用实地数据微调
预期提升	准确率提升, 适配本地光照条件和贡柑品种特征
技术风险	中等: 依赖去果园的时间窗口和天气条件

4.3 备用方案 (无法获取本地数据时)

如果去果园时间晚于 Week 2, 或老师/果农提供的照片数量不足, 启用以下备用方案:

1. **数据增强**：对 Kaggle 数据进行随机旋转（ $\pm 30^\circ$ ）、水平/垂直翻转、亮度/对比度/饱和度抖动，人工扩充 3-5 倍
2. **补充公开数据**：从 **PlantVillage 数据集**（50,000+ 张植物病害图）中筛选柑橘类图片
3. **调整定位**：第一版交付物定位为「柑橘病虫害识别」而非「贡柑专属」，但识别的病害本身就是贡柑的主要病害，在贡柑果园同样有效
4. **后续迭代**：拿到实地数据后在第二版中微调更新

第5章 设备需求

设备	已有/申请	用途	优
GPU 电脑	✓已有	模型训练	少
数码相机	建议申请	果园拍摄病害样本（画质比手机稳定）	
无人机	可申请（加分项用）	加分项「无人机模式」概念演示，航拍照片作大面积初筛	
激光设备	不建议申请	时间不足 + 硬件集成复杂 + 安全隐患	

5.1 关于无人机的补充说明

无人机用于两种场景：

- **加分项（Week 2 可实现）**：APP 中增加「无人机模式」入口，用户上传航拍远景照片，系统用颜色阈值分析（传统图像处理，不训练模型）标记异常黄色/褐色区域，演示「大面积初筛 → 精准定位」的概念
- **未来展望（不实际开发）**：无人机搭载摄像头实时视频流 + AI 识别，果园自动巡航发现异常自动报警。此功能需要飞控 SDK、实时推流、边缘计算等，超出两周范围

如果无人机不申请，加分项 2 可以用普通手机拍摄远景照片替代演示，不影响核心功能。

第6章 两人分工与两周时间线

6.1 角色划分

	A 同学：模型负责人	B 同学：APP 负责人
关键词	数据、训练、模型评估	前端、后端、产品体验
主要产出	可部署的 5 分类模型	可拍照识别的 Web APP
参考经验	曾用 PyTorch + XGBoost 完成黄斑水肿分类	需新学 Flask + HTML/CSS/JS
工作量	Week 1 重（跑实验），Week 2 轻（协助联调）	Week 1 轻（学习 + 搭框架），Week 2 重（联调部署）

6.2 Week 1（考试周，每天 ≤ 2 小时）

日期	A 同学（模型）	B 同学（APP）
一	下载 Kaggle Citrus Leaves 数据集；了解 5 种病害的视觉特征	安装 Flask，写出 Hello World；了解手机端网页如何调起相机（ <code><input type="file" accept="image/*" capture></code> ）
二	编写 <code>prepare.py</code> ：扫描数据集文件夹结构，生成 <code>labels.csv</code> （类别 → 数字标签）	学习 HTML/CSS 基础布局；在纸上画出 APP 页面草图（拍照页 → 结果页）
三	编写 <code>extract.py</code> ：ResNet-50 加载预训练权重 → 去掉全连接层 → 每张图提取 2048 维特征向量	写出前端第一版 HTML：静态页面，包含拍照按钮 + 图片预览区 + 结果占位区
四	晚上挂机跑特征提取（GPU）；检查输出的 <code>features.npz</code> 文件维度是否正确	学习 Flask 处理 POST 请求： <code>request.files</code> 接收图片，保存到临时目录

日期	A 同学（模型）	B 同学（APP）
五	编写 <code>train.py</code> : XGBoost 训练 + 5 折交叉验证 + 网格搜索超参; 处理样本不平衡 (SMOTE / <code>class_weight</code>)	后端 <code>/predict</code> 写空壳路由, 前后端串通: 上传图片 → 返回假 JSON 结果
六	分析第一轮训练结果: 准确率、混淆矩阵、各类别 <code>precision/recall</code>	查资料整理 5 类病害的防治建议文本, 写出 <code>disease_info.json</code> 数据文件
日	根据 CV 结果调参; 尝试 SMOTE 过采样; 记录 <code>baseline</code> 指标	前端完善: 拍照按钮 → 图片预览 → 加载动画 (CSS spinner) → 结果展示区
Week 1 里程碑: 模型有 baseline 准确率 ($\geq 80\%$), APP 有前后端骨架 (能跑通假数据)		

6.3 Week 2 (冲刺周, 全力投入)

日期	A 同学（模型）	B 同学（APP）
一	模型优化: 对比不同超参组合 (<code>n_estimators</code> , <code>max_depth</code> , <code>learning_rate</code>), 选最佳模型; 保存为 <code>.pkl</code>	把 A 训练好的模型文件 (<code>.pkl</code>) 接到 Flask 后端 <code>/predict</code> , 实现真实推理流程
二	编写 <code>predict.py</code> : 单张图片预测函数 (预处理 → 特征提取 → PCA → XGBoost 预测), 供 Flask 调用	前后端完整联调: 拍照 → 上传 → 模型推理 → 显示病害名称 + 置信度 + 防治建议
三	如果准确率不达标: 尝试 (a) 数据增强 (b) 替换为 EfficientNet-B0 端到端微调 (c) 合并相似类别降低难度	移动端适配: 在真实手机上测试, 调整按钮大小、字体、图片上传尺寸限制
四	协助联调; 处理模型推理时的异常: 图片格式不支持、尺寸过大、非叶片图片误拍	PWA 配置 (<code>manifest.json</code> + Service Worker): 让网页可以「添加到桌面」像原生 APP; 全流程 Bug 修复

日期	A 同学（模型）	B 同学（APP）
五	准备模型说明材料：方法概述、准确率、局限性	端到端测试：找 2-3 人用不同手机浏览器测试完整流程；写使用说明
六	缓冲：模型优化 / 补充训练数据	缓冲：界面打磨 / 处理边界情况（上传非图片文件、网络超时等）
日	最终检查 + 打包交付	
Week 2 里程碑：手机打开网页 → 拍照 → 2 秒内出病害名称 + 防治建议，流程完整可用		

6.4 分工并行度分析

阶段	并行度	说明
Week 1 前半	高	A 跑数据准备 + 模型，B 学 Flask + 写页面，互不依赖
Week 1 后半	高	A 跑训练（GPU 挂着），B 写后端空壳 + 前端页面
Week 2 前半	中	A 优化模型 + 导出，B 接模型到后端；需要协调接口格式
Week 2 后半	低	需要密切联调，两人坐在一起开发效率最高

第 7 章 加分项路线图

原则：核心功能（拍照识别）必须先完成，加分项在有余力时按顺序推进。

优先级	加分项	预估工时	实现方式
★1	防治建议含农药信息	半天	扩充 <code>disease_info.json</code> ：推荐农药名、稀释
★2	无人机巡航模式概念演示	1 天	APP 增加入口；用 OpenCV 颜色阈值分析航拍 不训练新模型，纯传统图
★3	识别历史记录	半天	JavaScript <code>localStorage</code> 存储历史：时间戳 列表页可回溯查看
★4	多角度综合判断	1-2 天	允许用户上传叶片 + 果实 + 树干三张照片，

7.1 未来展望（写在报告/答辩中，不实际开发）

- 无人机 + AI 实时视频流：无人机搭载摄像头自动巡航果园，实时视频流接入 AI 模型，发现异常自动报警并标记 GPS 坐标
- 激光驱虫：识别害虫种类和位置 → 激光瞄准 → 精准灭杀。参考日本大阪大学激光除虫研究（2023），该技术在实验室已可行，但工程化仍需时间
- 物联网联动：识别病害后自动生成施药方案，对接智能喷药无人机/机器人，形成「监测 → 诊断 → 治疗」闭环

第8章 风险分析与应对

风险	概率	影响	应对措施
Kaggle 数据下载困难	低	高	备选 PlantVillage 等替代数据集；也可用 Mendeley 原版数据集
模型准确率 <80%	中	高	□ 数据增强扩充样本 □ 换 EfficientNet 端到端微调 □ 类别合并降低难度
去果园时间晚于 Week 2	高	中	用 Kaggle 数据交付「柑橘病害识别」版本，后续微调作为持续优化
果园照片样本不足	中	中	数据增强 + 网上补充同类病害图片
B 同学零基础学 Flask 跟不上	中	高	Week 1 只要求跑通「假数据」；Week 2 A 同学协助联调；Flask 上手门槛低
GPU 内存不够 / 驱动问题	低	中	ResNet-50 量级较小，一般笔记本 GPU（4GB+）可跑；实在不行用 CPU（慢但能跑）

第9章 待与老师 / 队友确认的问题

1. 病虫害类别是否合适？

- 5 类够不够？是否需要增加特定病害（如贡柑黑腐病、褐腐病）？

- 是否需要加入虫害识别（如柑橘木虱、红蜘蛛）？虫害识别比病害分类复杂，需要解释清楚技术差异

2. 去果园的时间安排？

- 能否安排在 Week 1 周末或 Week 2 初？
- 如果只能 Week 3 去，Week 2 交付时模型暂定位为「柑橘病虫害识别」而非「贡柑专属」，老师能否接受？

3. 老师是否有存量病害照片？

- 实验室/合作果园是否有以前拍摄的贡柑病害照片可以先用？
- 哪怕每类只有 10–20 张，也能显著提升模型的针对性

4. 防治建议需要多详细？

- 基础版：病害简介 + 通用防治方法（3–5 条文字描述）
- 增强版：推荐农药名称 + 稀释比例 + 最佳喷施时间窗口
- 专业版：结合气象数据的分阶段防治日历

5. 无人机加分项的程度？

- 做一个「概念演示」模式（上传航拍照片 → 颜色分析标记异常区域）够不够？
- 还是必须接入真实无人机飞控？后者需要额外设备和飞控 SDK 开发时间

6. 分工是否需要调整？

- 上述 A/B 分工两人能否接受？（如两个人都想做模型，或都想做前端，可以调整）
- 如果一人主要做模型，另一人主要做 APP，工作量是否均衡？

第 10 章 下一步行动

确认上述问题后，立即可以启动：

1. 环境搭建（两人共同，30 分钟）

- 安装 Python 依赖: `pip install -r requirements.txt`
- 确认 GPU 驱动和 PyTorch CUDA 版本匹配

2. **A 同学**: 下载 Kaggle 数据集，跑通第一版训练（参考已有的 `train/` 目录代码）

3. **B 同学**: 学习 Flask（2 小时可跑通 Hello World + 图片上传），开始写前端页面

4. **两人**: 拉微信群/腾讯会议，每天同步 5 分钟进度

项目代码: `D:\gonggan-detection\`

编译引擎: XeLaTeX | 生成日期: 2026 年 7 月 3 日